

Project Coding – Frontend source (Next.js / TypeScript)

Seminar Hall Booking System – key Next.js/TS segments (full repo on GitHub)

Full source code – Included the most important code segments only (core schema, routing, JWT auth, request creation, department approval, overlap checks, and key UI logic). Omitted lines are marked with `// ...` or `-- ....`. Full sources are on GitHub. Repository: <https://github.com/sreejithr247/seminar-hall-booking-system>.

frontend/lib/api.ts

```
import axios from 'axios';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export const api = axios.create({
  baseURL: `${API_URL}/api`,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Attach JWT token from localStorage on every request
api.interceptors.request.use((config) => {
  if (typeof window !== 'undefined') {
    const token = localStorage.getItem('auth_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  }
  return config;
});

// Response interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Only redirect if we're on a protected page (not on login or public pages)
      if (typeof window !== 'undefined') {
        const publicPaths = ['/login', '/', '/halls'];
        const isPublic = publicPaths.some((p) =>
          window.location.pathname === p || window.location.pathname.startsWith(`${p}/`)
        );
        if (!isPublic) {
          localStorage.removeItem('auth_token');
          window.location.href = '/login';
        }
      }
    }
    return Promise.reject(error);
  }
);
```

frontend/lib/auth-context.tsx

```
'use client';

import { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { authApi } from '@lib/services';
import { User } from '@lib/types';

interface AuthContextType {
  user: User | null;
  requesterInfo: { requester_id: number; requester_type: string } | null;
  loading: boolean;
  logout: () => Promise<void>;
  refresh: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [requesterInfo, setRequesterInfo] = useState<{ requester_id: number; requester_type: string } | null>(null);
  const [loading, setLoading] = useState(true);

  const refresh = async () => {
    try {
      const res = await authApi.getMe();
      setUser(res.data.user);
      setRequesterInfo(res.data.requester ?? null);
    } catch {
      setUser(null);
      setRequesterInfo(null);
    } finally {
      setLoading(false);
    }
  };
};
```

```

const logout = async () => {
  try {
    await authApi.logout();
  } finally {
    localStorage.removeItem('auth_token');
    setUser(null);
    setRequesterInfo(null);
    window.location.href = '/login';
  }
};

useEffect(() => {
  refresh();
}, []);

return (
  <AuthContext.Provider value={{ user, requesterInfo, loading, logout, refresh }}>
    {children}
  </AuthContext.Provider>
);
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside AuthProvider');
  return ctx;
}

```

frontend/lib/types.ts (excerpt: L1-109)

```

// User and Auth Types
export type UserRole = 'admin' | 'dept_coordinator' | 'requester' | 'faculty';
export type ApprovalStatus = 'pending' | 'forwarded' | 'approved' | 'rejected' | 'amended';
export type BookingStatus = 'confirmed' | 'cancelled' | 'completed' | 'no_show';
export type RequesterType = 'class' | 'club';

export interface User {
  user_id: number;
  username: string;
  full_name: string;
  email?: string;
  phone?: string;
  role: UserRole;
  dept_id?: number;
  is_active: boolean;
  created_at: string;
  updated_at: string;
}

export interface Requester {
  requester_id: number;
  user_id: number;
  requester_type: RequesterType;
  class_id?: number;
  club_id?: number;
  created_at: string;
}

export interface Hall {
  hall_id: number;
  hall_name: string;
  capacity: number;
  location?: string;
  facilities: Record<string, any>;
  is_active: boolean;
}

export interface Request {
  request_id: number;
  requester_id: number;
  hall_id: number;
  hall_name?: string;
  hall_location?: string;
  hall_capacity?: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
  dept_status: ApprovalStatus;
  dept_approved_by?: number;
  dept_approved_at?: string;
  dept_remarks?: string;
  admin_status: ApprovalStatus;
  admin_approved_by?: number;
  admin_approved_at?: string;
  admin_remarks?: string;
  requested_at: string;
  is_cancelled: boolean;
}

```

```

}

export interface Booking {
  booking_id: number;
  request_id: number;
  hall_id: number;
  requester_id: number;
  event_title: string;
  event_date: string;
  start_time: string;
  end_time: string;
  status: BookingStatus;
  created_at: string;
  completed_at?: string;
}

export interface AvailabilitySlot {
  hall_id: number;
  hall_name: string;
  date: string;
  bookings: Booking[];
}

export interface LoginRequest {
  username: string;
  password: string;
}

export interface LoginResponse {
  user: User;
  token: string;
}

export interface CreateRequestInput {
  hall_id: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
}

export interface ReviewRequestInput {
  action: 'approve' | 'reject';
  remarks?: string;
}
// ...

```

frontend/lib/services.ts (excerpt: L1-30, L70-84)

```

import { api } from './api';
import {
  Hall, Booking, Department, Class, Club,
  HallUsageReport, DepartmentUsageReport,
  Request, User, ReviewRequestInput, CreateRequestInput,
} from './types';

// — AUTH —————

export const authApi = {
  login: async (username: string, password: string) => {
    const res = await api.post<{ user: User; token: string }>('/auth/login', { username, password });
    // Save token to localStorage so the request interceptor can use it
    if (typeof window !== 'undefined') {
      localStorage.setItem('auth_token', res.data.token);
    }
    return res;
  },

  logout: () => api.post('/auth/logout'),

  getMe: () => api.get<{ user: User; requester?: { requester_id: number; requester_type: string } }>('/auth/me'),

  /** Any logged-in user: verify current password and set a new one */
  changeOwnPassword: (currentPassword: string, newPassword: string) =>
    api.patch('/auth/password', {
      current_password: currentPassword,
      new_password: newPassword,
    }),
};
// ...
// — REQUESTS —————

export const requestsApi = {
  // Requester
  create: (data: CreateRequestInput) => api.post<Request>('/requests', data),
  getMy: () => api.get<Request[]>('/requests/my'),

  // Coordinator
  getDeptPending: () => api.get<Request[]>('/requests/dept-pending'),

```

```

    deptReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/dept-review`, payload),

    // Admin
    getAdminPending: () => api.get<Request[]>(`/requests/admin-pending`),
    adminReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/admin-review`, payload),
  };
  // ...

```

frontend/app/login/page.tsx (excerpt: L1-45)

```

'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { authApi } from '@lib/services';
import { useAuth } from '@lib/auth-context';
import { toast } from 'sonner';

export default function LoginPage() {
  const router = useRouter();
  const { user, loading, refresh } = useAuth();
  const [formData, setFormData] = useState({ username: '', password: '' });
  const [submitting, setSubmitting] = useState(false);

  // If already logged in, redirect to dashboard
  useEffect(() => {
    if (!loading && user) {
      router.push('/dashboard');
    }
  }, [user, loading, router]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitting(true);
    try {
      await authApi.login(formData.username, formData.password);
      // Refresh AuthContext so user state is populated BEFORE navigating
      await refresh();
      toast.success('Login successful!');
      router.push('/dashboard');
    } catch (error: any) {
      toast.error(error.response?.data?.error || 'Login failed');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 to-white">
      <div className="bg-white p-8 rounded-lg shadow-lg w-full max-w-md">
        <h1 className="text-3xl font-bold text-blue-900 mb-6 text-center">
          Login
        </h1>
        <form onSubmit={handleSubmit} className="space-y-4">

```

frontend/app/requester/request-new/page.tsx (excerpt: L1-55)

```

'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@lib/auth-context';
import { hallsApi, requestsApi } from '@lib/services';
import { Hall } from '@lib/types';
import { toast } from 'sonner';

export default function NewRequestPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [halls, setHalls] = useState<Hall[]>([]);
  const [submitting, setSubmitting] = useState(false);

  const [form, setForm] = useState({
    hall_id: 0,
    event_title: '',
    event_description: '',
    event_date: '',
    start_time: '',
    end_time: '',
    expected_attendees: '',
    purpose: '',
  });

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'requester') router.push('/dashboard');
  }, [user, loading, router]);

  useEffect(() => {
    hallsApi.getAll().then((res) => setHalls(res.data)).catch(() => toast.error('Failed to load halls'));
  }, []);

```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!form.hall_id) { toast.error('Please select a hall'); return; }

  setSubmitting(true);
  try {
    await requestsApi.create({
      hall_id: form.hall_id,
      event_title: form.event_title,
      event_description: form.event_description || undefined,
      event_date: form.event_date,
      start_time: form.start_time,
      end_time: form.end_time,
      expected_attendees: form.expected_attendees ? parseInt(form.expected_attendees) : undefined,
      purpose: form.purpose || undefined,
    });
    toast.success('Request submitted successfully!');
    router.push('/requester/my-requests');
  } catch (err: any) {
    toast.error(err.response?.data?.error || 'Failed to submit request');
  }
}
// ...

frontend/app/coordinator/pending/page.tsx (excerpt: L1-50)
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@lib/auth-context';
import { requestsApi } from '@lib/services';
import { Request } from '@lib/types';
import { formatDate, formatTime } from '@lib/utils';
import { toast } from 'sonner';

export default function CoordinatorPendingPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [requests, setRequests] = useState<Request[]>([]);
  const [fetching, setFetching] = useState(true);
  const [reviewing, setReviewing] = useState<number | null>(null);

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'dept_coordinator') router.push('/dashboard');
  }, [user, loading, router]);

  const fetchRequests = () => {
    requestsApi.getDeptPending()
      .then((res) => setRequests(res.data))
      .catch(() => toast.error('Failed to load requests'))
      .finally(() => setFetching(false));
  };

  useEffect(() => {
    if (user?.role === 'dept_coordinator') fetchRequests();
  }, [user]);

  const handleReview = async (id: number, action: 'approve' | 'reject') => {
    const remarks = action === 'reject' ? window.prompt('Enter rejection remarks (optional):') ?? undefined :
    undefined;
    setReviewing(id);
    try {
      await requestsApi.deptReview(id, { action, remarks });
      toast.success(`Request ${action === 'approve' ? 'forwarded to admin' : 'rejected'}`);
      fetchRequests();
    } catch (err: any) {
      toast.error(err.response?.data?.error || 'Action failed');
    } finally {
      setReviewing(null);
    }
  };

  if (loading || fetching) return <div className="min-h-screen flex items-center justify-center"><div
  className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div></div>;

  return (
    // ...

frontend/app/halls/[id]/calendar/page.tsx (excerpt: L1-68)
'use client';

import { useState, useEffect } from 'react';
import { useParams, useRouter } from 'next/navigation';
import Link from 'next/link';
import { Calendar, momentLocalizer, View } from 'react-big-calendar';
import moment from 'moment';
import 'react-big-calendar/lib/css/react-big-calendar.css';
import { api } from '@lib/api';
import { Booking, Hall } from '@lib/types';
import { formatTime } from '@lib/utils';
import { toast } from 'sonner';

const localizer = momentLocalizer(moment);

```

```

export default function HallCalendarPage() {
  const params = useParams();
  const router = useRouter();
  const hallId = params.id as string;
  const [hall, setHall] = useState<Hall | null>(null);
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedDate, setSelectedDate] = useState(new Date());
  const [view, setView] = useState<View>('day');

  useEffect(() => {
    fetchHall();
  }, [hallId]);

  useEffect(() => {
    fetchBookings();
  }, [hallId, selectedDate]);

  const fetchHall = async () => {
    if (!hallId) return;
    try {
      const response = await api.get(`/halls/${hallId}`);
      setHall(response.data);
    } catch (error: any) {
      toast.error('Failed to fetch hall details');
    }
  };

  const fetchBookings = async () => {
    if (!hallId) return;
    try {
      // Fetch for the whole month surrounding selectedDate to be safe regardless of view
      const startOfMonth = moment(selectedDate).startOf('month').format('YYYY-MM-DD');
      const endOfMonth = moment(selectedDate).endOf('month').format('YYYY-MM-DD');

      const response = await api.get(`/availability?
start_date=${startOfMonth}&end_date=${endOfMonth}&hall_id=${hallId}`);
      setBookings(response.data || []);
    } catch (error) {
      toast.error('Failed to fetch bookings');
    } finally {
      setLoading(false);
    }
  };

  const events = (bookings || []).map((booking) => {
    const datePart = booking.event_date.split('T')[0];
    return {
      title: booking.event_title,
      start: moment(`${datePart} ${booking.start_time}`).toDate(),
      end: moment(`${datePart} ${booking.end_time}`).toDate(),
      resource: booking,
    };
  });
  // ...

```